



# Nuevos paradigmas de Base de Datos

# Taller ETL Pipeline usando Python, MySQL y Airflow (VS Code)

## 1. Objetivos del Taller

Los estudiantes aprenderán a:

- Conectarse a una base de datos MySQL desde Python.
- Extraer datos usando SQL y pandas.
- Transformar los datos localmente.
- Integrar datos desde un archivo CSV externo.
- Automatizar el proceso ETL usando Airflow.
- Ejecutar todo desde Visual Studio Code.

## 2. Requisitos Previos

Haber completado todas las guías y talleres de la Unidad 3 (ETL).

Herramientas necesarias:

- Docker Desktop
- Visual Studio Code
- Python 3.11
- Plugin de Python en VS Code
- Git (opcional)

## 3. Estructura del Proyecto

```
paradigma/
├── airflow/
│   └── etl_dag.py          # DAG de Airflow
├── mysql/
│   ├── db_movies_neflix_transact.sql
│   ├── data_warehouse_netflix.sql
│   └── data/
│       └── Awards_movie.csv
├── python/
│   └── etl_pipeline.py    # Script ETL en Python
└── docker-compose.yml
```

#### 4. Verificar Contenedor MySQL

Ejecuta en terminal:

```
docker ps
```

```
PS C:\Users\Administrador\Documents\GitHub\paradigmas> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
e6f7d542de3e   mysql:8.0      "docker-entrypoint.s..." 48 minutes ago Up 48 minutes 33060/tcp, 0.0.0.0:3310->3306/tcp   paradigmas-mysql-1
54655672df22   apache/airflow:2.8.1-python3.11 "/usr/bin/dumb-init ..." 48 minutes ago Up 48 minutes 0.0.0.0:8080->8080/tcp             airflow
878cb91daf30   mongo:6.0      "docker-entrypoint.s..." 48 minutes ago Up 48 minutes 0.0.0.0:27017->27017/tcp           paradigmas-mongodb-1
```

Si ves el contenedor paradigmas-mongodb-1 en la lista, entonces está corriendo correctamente.

#### Conectarte al contenedor MySQL

El contenedor **MySQL** está corriendo en el puerto 3310, y para acceder a él, puedes usar el siguiente comando.

1. Conéctate al contenedor MySQL con el terminal de VsC:

```
docker exec -it paradigmas-mysql-1 bash
```

```
PS C:\Users\Administrador\Documents\GitHub\paradigmas> docker exec -it paradigmas-mysql-1 bash
bash-5.1#
```

Conéctate desde VS Code:

Una vez dentro del contenedor, ejecuta el cliente de MySQL: Ingresa la contraseña **root** (si configuraste esa contraseña en el docker-compose.yml).

```
mysql -u root -p
```

(Ingresa la contraseña configurada en docker-compose.yml inicialmente es root)

```
bash-5.1# mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.41 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
```

Ahora ya estás dentro de la interfaz de MySQL. Puedes listar las bases de datos:

```
SHOW DATABASES;
```



```

+-----+
| airflow
| db_movies_netflix_transact
| dw_netflix
| information_schema
| mysql
| performance_schema
| sys
+-----+
7 rows in set (0.00 sec)

```

## 5. Parte I: Script Python ETL

Puedes descargarlo desde: [etl\\_MYSQL.ipynb](https://raw.githubusercontent.com/adiacla/bigdata/refs/heads/master/etl_MYSQL.ipynb)

[https://raw.githubusercontent.com/adiacla/bigdata/refs/heads/master/etl\\_MYSQL.ipynb](https://raw.githubusercontent.com/adiacla/bigdata/refs/heads/master/etl_MYSQL.ipynb)

Este contiene el ejemplo de ETL Pipeline en Python.

The screenshot shows a Jupyter Notebook titled 'etl\_pipeline.ipynb' in a web browser. The left sidebar shows the file explorer with 'etl\_pipeline.ipynb' selected. The main area displays the following Python code:

```

python > etl_pipeline.ipynb
Generate + Code + Markdown ▶ Run All ⌵ Clear All Outputs | Outline ...

import pandas as pd
import sqlalchemy as db

# Conexión a la base de datos MySQL (expuesta en el puerto 3310)
engine = db.create_engine("mysql://root:root@127.0.0.1:3310/db_movies_netflix_transact")
conn = engine.connect()

# 1. EXTRAER: Datos desde MySQL
query = """
SELECT
    movie.movieID as movieID, movie.movieTitle as title, movie.releaseDate as releaseDate,
    gender.name as gender , person.name as participantName, participant.participantRole as rolepart
FROM movie
INNER JOIN participant ON movie.movieID=participant.movieID
INNER JOIN person ON person.personID = participant.personID
INNER JOIN movie_gender ON movie.movieID = movie_gender.movieID
INNER JOIN gender ON movie_gender.genderID = gender.genderID
"""

movies_data = pd.read_sql(query, con=conn)
movies_data["movieID"] = movies_data["movieID"].astype(int)

# 2. CARGAR CSV

```

## 6. Parte II: Automatizar ETL con Airflow

Apache Airflow permite definir flujos de trabajo como DAGs (Grafos Acíclicos Dirigidos), que pueden ejecutarse manual o automáticamente.

Este ejemplo muestra un DAG básico en Airflow que ejecuta los pasos extract, transform y load.

### Objetivo del DAG etl\_dependencias

- **mostrar\_fecha\_hora:** tarea Bash que imprime la fecha y hora.
- **tarea\_1 y tarea\_2:** tareas Python que corren en paralelo.
- **tarea\_3:** se ejecuta solo cuando las 2 tareas finalizan.
- **Se ejecuta automáticamente todos los días a las 12:00 a.m.**

### Ejemplo de DAG en Python (Airflow)

```
from airflow import DAG
from airflow.operators.bash import BashOperator
from airflow.operators.python import PythonOperator
from datetime import datetime
import pandas as pd
import pymysql

# Conexión MySQL (desde dentro del contenedor Airflow)
def get_connection():
    return pymysql.connect(
        host='host.docker.internal',
        port=3310,
        user='root',
        password='root',
        database=db_movies_netflix_transact'
    )

# Tarea 1: ETL de tabla de películas
def etl_transacciones():
    movies_award=pd.read_csv("/data/Awards_movie.csv")
    movies_award['movieID']=movies_award['movieID'].astype('int')
    movies_award.rename(columns={'Aware':'Award'}, inplace=True)
    movies_award.to_csv('/data/etl_award.csv', index=False)

# Tarea 2: ETL del DW
def etl_dw():
    conn = get_connection()
    query = """
    SELECT
```

```

        movie.movieID as movieID, movie.movieTitle as title, movie.releaseDate as releaseDate,
        gender.name as gender, person.name as participantName, participant.participantRole as
        roleparticipant
    FROM movie
    INNER JOIN participant
    ON movie.movieID=participant.movieID
    INNER JOIN person
    ON person.personID = participant.personID
    INNER JOIN movie_gender
    ON movie.movieID = movie_gender.movieID
    INNER JOIN gender
    ON movie_gender.genderID = gender.genderID
    """

    movies_data=pd.read_sql(query, con=conn)
    movies_data["movieID"]=movies_data["movieID"].astype('int')
    movies_data.to_csv('/data/etL_transacciones.csv', index=False)
    conn.close()

# Tarea 3: combinación de resultados (solo depende de tarea_2)
def combinar_etl():
    df1 = pd.read_csv('/data/etL_transacciones.csv')
    df2 = pd.read_csv('/data/etL_award.csv')
    df_final = pd.concat([df1, df2], axis=1)
    df_final.to_csv('/data/etL_combinado.csv', index=False)

# Definición del DAG
with DAG(
    dag_id="etL_dependencias",
    description="ETL por etapas con dependencias en Airflow",
    start_date=datetime(2024, 4, 1),
    schedule_interval="0 0 * * *", # se ejecuta todos los días a las 00:00
    catchup=False,
    tags=['ETL', 'Airflow', 'MySQL']
) as dag:

    mostrar_fecha_hora = BashOperator(
        task_id="mostrar_fecha_hora",
        bash_command="echo 'Hora actual:' && date"
    )

    tarea_1 = PythonOperator(
        task_id="etL_transacciones",
        python_callable=etL_transacciones
    )

    tarea_2 = PythonOperator(
        task_id="etL_dw",
        python_callable=etL_dw
    )

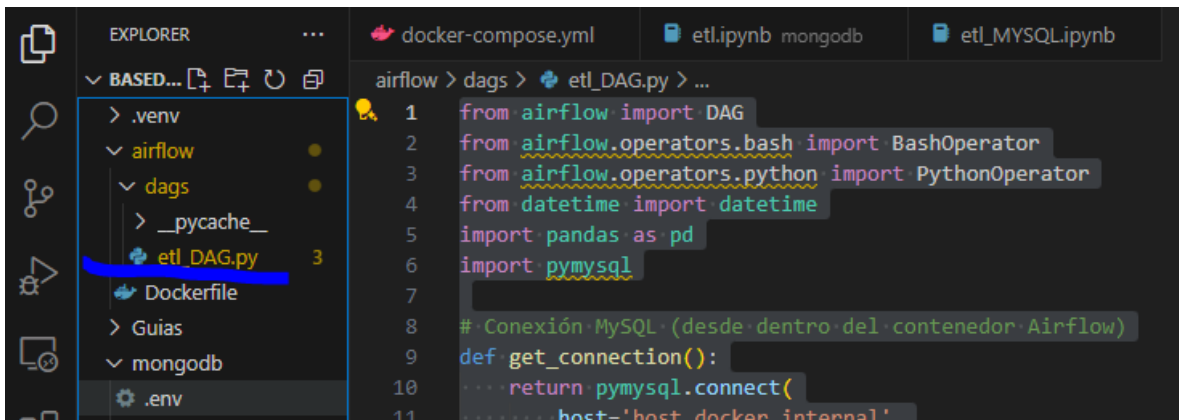
    tarea_3 = PythonOperator(
        task_id="combinar_datos",

```

```
python_callable=combinar_etl
)

# Dependencias
mostrar_fecha_hora >> [tarea_1, tarea_2]
tarea_1 >> tarea_3
tarea_2 >> tarea_3
```

Este archivo lo puede descargar de [etl\\_DAG.py](#), lo debe guardar en una carpeta en airflow/dag, de esta manera.



## 7. Ejecutar el DAG en Airflow

### Paso 1: Verifica que Docker esté corriendo

```
docker ps
```

### Paso 2: Accede a Airflow

- Navega a: <http://localhost:8080>
- Usuario: admin
- Contraseña: admin
- Busca el DAG etl\_dependencias y haz clic en el botón ▶ para ejecutarlo manualmente.

localhost:8080/login/

Airflow

21

Sign In

Enter your login and password below:

Username:

airflow

Password:

.....

Sign In

### Paso 3: Ejecutar el DAG

Do not use **SQLite** as metadata DB in production – it should only be used for dev/testing. We recommend using Postgres or MySQL. [Click here](#) for more information.

Do not use the **SequentialExecutor** in production. [Click here](#) for more information.

#### DAGs

All 1 Active 0 Paused 1 Running 0 Failed 0 Filter DAGs by tag Search DAGs Auto-refresh

| DAG                  | Owner   | Runs | Schedule | Last Run | Next Run | Recent Tasks | Actions                                                                                                                                                                         | Links |
|----------------------|---------|------|----------|----------|----------|--------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| etl_netflix_pipeline | airflow | 0    | None     |          |          |              |   ... |       |

- Activa el DAG etl\_netflix\_pipeline.
- Haz clic en  (Trigger DAG).
- Verifica que el archivo resultados\_completos.csv se genere en mysql/data/.